

## INDEX

| Practical No. | Practical                                          | Date     | Page No. | Sign |
|---------------|----------------------------------------------------|----------|----------|------|
| 1             | Perform Geometric Transformation                   | 04/03/24 | 2        |      |
| 2             | Perform Image Stitching                            | 09/03/24 | 5        |      |
| 3             | Perform Camera Calibration                         | 11/03/24 | 8        |      |
| 4             | A Perform Face Detection                           | 16/03/24 | 11       |      |
|               | B Perform Object Detection                         | 18/03/24 | 13       |      |
|               | C Perform Pedestrian Detection                     | 23/03/24 | 16       |      |
|               | D Perform Face Recognition                         | 30/03/24 | 18       |      |
| 5             | Construct 3D Model from Images                     | 01/04/24 | 20       |      |
| 6             | Implement Object Detection and Tracking from Video | 08/04/24 | 23       |      |
| 7             | Perform Feature Extraction using RANSAC            | 15/04/24 | 25       |      |
| 8             | Perform Colorization                               | 25/04/24 | 27       |      |
| 9             | Perform Text Detection and Recognition             | 02/05/24 | 29       |      |
| 10            | Perform Image Matting and Composting               | 09/05/24 | 31       |      |

## Practical 1

**Aim: - Perform Geometric Transformations**

### Theory:

Often, during image processing, the geometry of the image (such as the width and height of the image) needs to be transformed. This process is called geometric transformation, images are nothing but matrices.

Here are the geometric transformations that we are performing in this practical:

- **Translation:** Translation basically means shifting the object's location. It means shifting the object horizontally or vertically by some defined off-set (measured in pixels). Each pixel is shifted 100 pixels in the x-direction and each pixel is shifted 80 pixels in the y-direction.
- **Rotation:** This technique rotates an image by a specified angle and by the given axis or point. It performs a geometric transform which maps the position of a point in current image to the output image by rotating it by the user-defined angle through the specified axis. The points that lie outside the boundary of an output image are ignored.
- **Image scaling or resizing:** Scaling means resizing an image which means an image is made bigger or smaller in x- or/and y-direction.
- **Affine transformation:** An affine transformation is a transformation that preserves collinearity and the ratio of distances (for example – the midpoint of a line segment is still the midpoint even after the transformation))  
The parallel lines in an original image will be parallel in the output image.  
In general, an affine transformation is a composition of translations, rotations, shears, and magnifications.
- **Perspective transformation:** Perspective transformation, also known as homography, is a projective transformation that maps points in one plane to another while preserving straight lines. It can account for changes in perspective and orientation, making it suitable for applications like image stitching, rectifying images, and augmented reality. In computer vision, perspective transformation is often represented as a  $3 \times 3$  matrix called the homography matrix. When applied to an image, this matrix can correct perspective distortions, align images from different viewpoints, and create panoramas.

### Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load on image
image = cv2.imread('left1.jpg')
# Get image dimensions
height, width = image.shape[:2]
# Display the original image
```

```

plt.figure(figsize=(6,6))
plt.subplot(2,3,1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')
plt.show()
# Define the translation matrix
translation_matrix = np.float32([[1,0,300],[0,1,100]])
# Apply translation transformation
translated_image = cv2.warpAffine(image, translation_matrix, (width,height))
# Display the translated image
plt.figure(figsize=(6,6))
plt.subplot(2,3,2)
plt.imshow(cv2.cvtColor(translated_image, cv2.COLOR_BGR2RGB))
plt.title("Translated Image")
plt.axis('off')
plt.show()
# Define the rotation matrix
rotation_matrix = cv2.getRotationMatrix2D((width/2, height/2), 60, 1)
# Apply rotation transformation
rotated_image = cv2.warpAffine(image, rotation_matrix, (width,height))
# Display the rotated image
plt.figure(figsize=(6,6))
plt.subplot(2,3,3)
plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB))
plt.title('Rotated Image')
plt.axis('off')
plt.show()
# Define the scaling matrix
scaling_matrix = np.float32([[0.8,0,0],[0,0.8,0]])
# Apply Scaling transformation
scaled_image = cv2.warpAffine(image, scaling_matrix, (int(width*0.5), int(height*0.5)))
# Display the scaled image
plt.figure(figsize=(6,6))
plt.subplot(2,3,4)
plt.imshow(cv2.cvtColor(scaled_image, cv2.COLOR_BGR2RGB))
plt.title('Scaled Image')
plt.axis('off')
plt.show()
# Define affine transformation points
original_points = np.float32([[50,50], [200,50], [50,200]])
transformed_points = np.float32([[10,100], [200,50], [100,250]])
# Compute the affine transformation matrix
affine_matrix = cv2.getAffineTransform(original_points, transformed_points)
# Apply affine transformation
affine_transformed_image = cv2.warpAffine(image, affine_matrix, (width,height))
# Display the affine transformed image

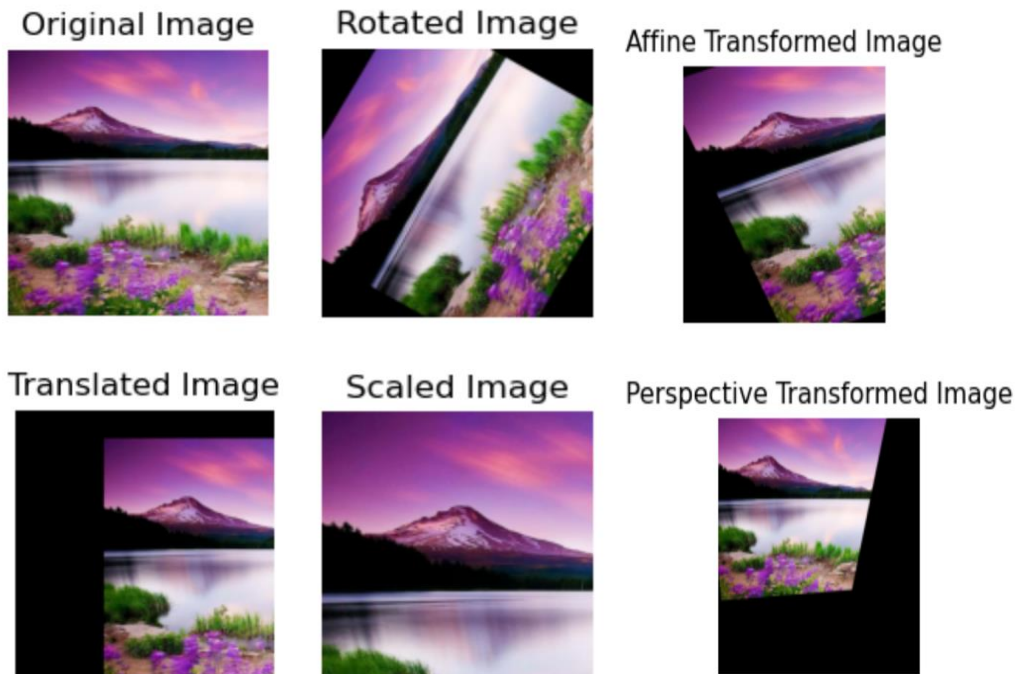
```

```

plt.figure(figsize=(6,6))
plt.subplot(2,3,5)
plt.imshow(cv2.cvtColor(affine_transformed_image, cv2.COLOR_BGR2RGB))
plt.title('Affine Transformed Image')
plt.axis('off')
plt.show()
# Define perspective transformation points
original_points = np.float32([[56,65], [368,52], [28,387], [389,390]])
transformed_points = np.float32([[0,0], [300,0], [0,300], [300,300]])
# Compute the perspective transformation matrix
perspective_matrix = cv2.getPerspectiveTransform(original_points, transformed_points)
# Apply perspective transformation
perspective_transformed_image = cv2.warpPerspective(image, perspective_matrix,
(width,height))
# Display the perspective transformed image
plt.figure(figsize=(6,6))
plt.subplot(2,3,6)
plt.imshow(cv2.cvtColor(perspective_transformed_image, cv2.COLOR_BGR2RGB))
plt.title('Perspective Transformed Image')
plt.axis('off')
plt.show()

```

### Output:



## Practical 2

### Aim: - Perform Image Stitching

#### Theory:

Image stitching is the process of combining multiple images to create a single larger image, often referred to as a panorama. The images depict the same three-dimensional scene, and they must overlap in some regions. Image stitching aims to create a seamless transition between adjacent images while preserving the geometry and visual quality.

Image stitching is a multi-step technique that involves the following image-processing operations:

- **Feature detection:** Feature detection is the task of identifying distinctive and salient points in an image. These points, generally called key points, serve as landmarks for aligning the acquired images.
- **Feature matching:** The goal is to identify points in one image that correspond to the same real-world location in another image. For each detected feature, a descriptor is computed. It captures the key characteristics of the feature and is used for matching.
- **Transformation estimation:** Once the pairs of matching features are identified, the transformations that align each image with a reference image are estimated. Such transformation is called homography.
- **Image warping:** This step involves applying the transformations found in the previous step to all the images, except the reference image. In this way, the images are aligned in the same reference system.
- **Blending:** Uneven lighting conditions and exposure differences between the acquired images lead to visible seams in the final panorama. Image blending techniques allow us to mitigate the seam problem.

#### Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load images
img_ = cv2.imread('right1.jpg')
img1 = cv2.cvtColor(img_, cv2.COLOR_BGR2GRAY)
img = cv2.imread('left1.jpg')
img2 = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

#Initialize SIFT detector
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

# Create a BFMatcher object with distance measurement cv2.NORM_L2
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck = False)

# Perform the matching between the SIFT descriptors of the images
```

```

matches = bf.knnMatch(des1,des2,k=2)

# Apply the ratio test to find good matches
good = []
for m,n in matches:
    if m.distance < 0.75*n.distance:
        good.append(m)

# Atleast 4 matches are to be there to find the homography
if len(good)>4:
    # prepare source and destination points
    src_pts = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)

    # Compute Homography
    H, status = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

    # Use homography to warp image
    dst = cv2.warpPerspective(img_,H, (img.shape[1]+img_.shape[1],img.shape[0]))

    # Convert warped image from BGR to RGB for matplotlib
    dst_rgb = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)

    # Display the warped image
    plt.subplot(122), plt.imshow(dst_rgb), plt.title('Warped Image')
    plt.show()

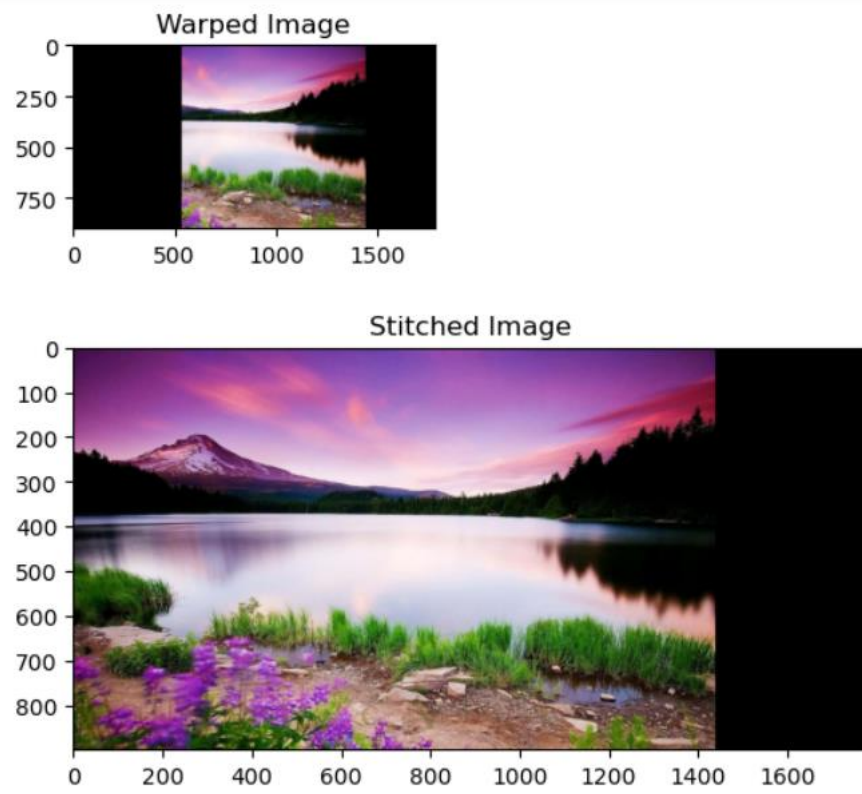
    # Place the left image on the appropriate position
    dst[0:img.shape[0], 0:img.shape[1]] = img

    # Convert the combined image from BGR to RGB for matplotlib
    combined_rgb = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
    # Save the stitched image as output.jpg in the BGR format
    cv2.imwrite('output.jpg',dst)
    # Display the stitched image
    plt.imshow(combined_rgb)
    plt.title('Stitched Image')
    plt.show()

else:
    raise AssertionError('Not enough matches are found - {}/{}'.format(len(good),4))

```

## Output:



## Practical 3

### Aim: - Perform Camera Calibration

#### Theory:

The process of computing the camera parameters is called camera calibration. Generally, the camera calibration process uses images of a 3D object with a geometrical pattern (e.g. checker board). The pattern is called the calibration grid. The 3D co-ordinates of the pattern are matched to 2D image points. The main function of camera calibration is to remove distortions caused by camera lenses and accurately relate image coordinates to real-world measurements.

The camera calibration procedure determines which light source is associated with each pixel on the resulting image. Since the camera used in the process is a complex CCD camera, errors resulting from the misaligned lenses and deformations in their structures result in more complex distortions in the acquired image.

Camera calibration is crucial in the photogrammetric process for several reasons:

- Accuracy: Calibration helps ensure accurate measurements by accounting for lens distortions and other optical imperfections in the camera system.
- Precision: It enables precise reconstruction of 3D scenes from 2D images by providing accurate parameters for the camera's internal geometry.
- Consistency: Calibrated cameras yield consistent results, which is essential for applications such as 3D modeling, surveying, and remote sensing.
- Compatibility: Calibrated cameras can be used interchangeably, as their parameters are known and can be easily accounted for in the photogrammetric workflow.
- Quality Control: It allows for quality control and validation of the captured images, ensuring that they meet the required standards for accuracy and reliability.

#### Code:

```
import numpy as np
import cv2
import glob
import matplotlib.pyplot as plt

# termination criteria
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# prepare object points, like (0,0,0), (1,0,0), (2,0,0) ....,(6,5,0)
objp = np.zeros((6*9,3), np.float32)
objp[:, :2] = np.mgrid[0:9,0:6].T.reshape(-1,2)

# Arrays to store object points and image points from all the images.
objpoints = [] # 3d point in real world space
imgpoints = [] # 2d points in image plane.

images = glob.glob('left/*.jpg') #read a series of images
```

```

for fname in images:
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) #Convert the image to gray

    # Find the chess board corners
    ret, corners = cv2.findChessboardCorners(gray, (9,6), None)

    # If found, add object points, image points (after refining them)
    if ret == True:
        objpoints.append(objp)

        corners2=cv2.cornerSubPix(gray, corners, (11,11), (-1,-1), criteria) #refine the corner
locations
        imgpoints.append(corners2)

        # Draw and display the corners

        cv2.drawChessboardCorners(img, (9,6), corners2, ret)
        cv2.imshow('img', img)
        cv2.waitKey(500)
cv2.destroyAllWindows()

#calibration
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints, gray.shape[:-1],None,None)

img = cv2.imread('left/left12.jpg')
h, w = img.shape[:2]
newcameramtx, roi = cv2.getOptimalNewCameraMatrix(mtx, dist, (w,h), 1, (w,h))

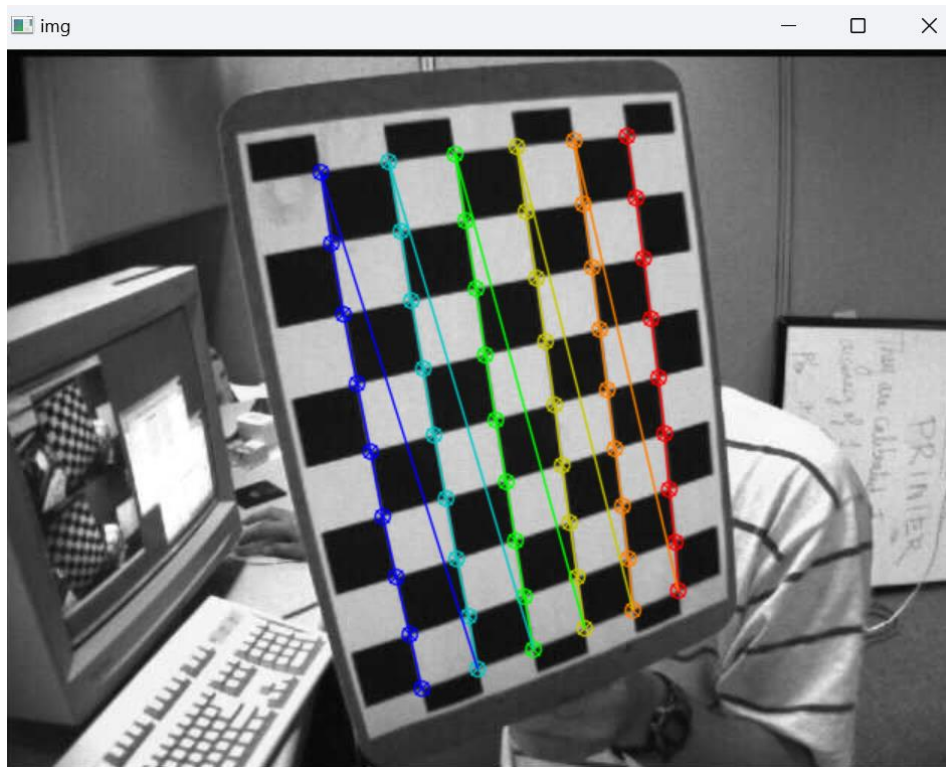
# undistort
dst = cv.undistort(img, mtx, dist, None, newcameramtx)

# crop the image
x, y, w, h = roi
dst = dst[y:y+h, x:x+w]

# Plot original and undistorted images
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
ax[0].set_title('Original Image')
ax[0].axis('off')
ax[1].imshow(cv2.cvtColor(dst, cv2.COLOR_BGR2RGB))
ax[1].set_title('Undistorted Image')
ax[1].axis('off')
plt.show()

```

## Output:



Original Image



Undistorted Image



## Practical 4

### Aim: - Perform Face Detection

#### Theory:

Face detection, also called facial detection, is an artificial intelligence (AI)-based computer technology used to find and identify human faces in digital images and video. Face detection technology is often used for surveillance and tracking of people in real time.

There are some steps that how face detection operates, which are as follows:

1. Firstly, desired picture is transformed from RGB to Grayscale because it is easy to detect faces in the grayscale.
2. The segments are created, which is used for contour detection or segments the multiple objects in a single image so that the classifier can quickly detect the objects and faces in the picture.
3. Using Haar-Like features algorithm (used for feature selection or feature extraction for an object), which is proposed by Viola and Jones the location of the human faces in a frame or image is found.
4. Lastly, the coordinates of x, y, w, h which makes a rectangle box in the picture to show the location of the face.

#### a) Image Face Detection

##### Code:

```
!pip install open-cv python
```

```
# detect face from input image and save it on the disk.  
import cv2
```

```
# Load the pre-trained Haar Cascade model for face detection  
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +  
'haarcascade_frontalface_default.xml')
```

```
# load the image where you want to detect face  
image_path = 'D:/MSCIT_books/sem2/img1.jpeg' # path to your image  
image = cv2.imread(image_path)
```

```
# convert the image to grayscale  
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# detect faces in the image  
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,  
minSize=(30,30))
```

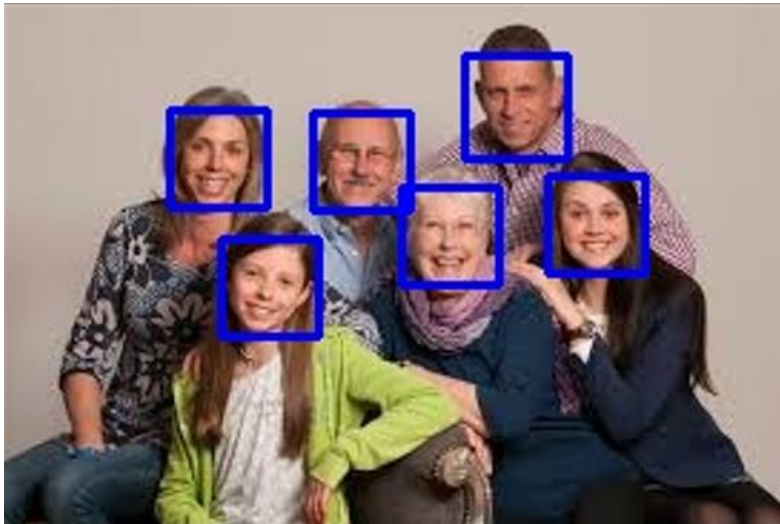
```
# draw rectangles around each face  
for (x,y,w,h) in faces:  
    cv2.rectangle(image,(x,y),(x+w,y+h),(255,0,0),2)
```

```
# save the image with faces highlighted
output_path = 'D:/MSCIT_books/sem2/faces_detected_1.jpg'

# corrected file extension
cv2.imwrite(output_path, image)

print(f"Faces Detected: {len(faces)}. Output saved to {output_path}")
```

**Output:**



## **Aim: - Perform Object Detection**

### **Theory:**

Object detection, within computer vision, involves identifying objects within images or videos. Object detection is merely to recognize the object with bounding box in the image, where in image classification, we can simply categorize(classify) that is an object in the image or not in terms of the likelihood (Probability). It is like object recognition, which involves identifying the object category, but object detection goes a step further and localizes the object within the image.

Object detection applications can be performed using two different data analysis techniques:

A. Image Processing: In image processing, the algorithm analyses an image to detect objects based on certain features, such as edges or textures. This approach typically involves applying a series of image processing techniques, such as filtering, thresholding, and segmentation, to extract regions of interest in the image. These regions are then analysed to determine whether they contain an object or not.

B. Deep Neural Network: In a deep neural network, the algorithm is trained on large, labelled datasets to detect objects. This approach involves using a convolutional neural network (CNN) architecture, which is a type of deep learning algorithm specifically designed for image analysis. The network learns to detect objects by analysing features of the image at different levels of abstraction, using multiple layers of artificial neurons to perform increasingly complex analyses. The output of the network is a bounding box around the detected object, along with a confidence score that represents how certain the network is that the object is present.

### **Code:**

```
import argparse
import numpy as np
import cv2
import matplotlib.pyplot as plt
args = argparse.Namespace(
    image="cat.jpeg",
    weights="yolov3.weights",
    config="yolov3.cfg",
    classes="yolov3.txt"
)
def get_output_layers(net):
    layer_names = net.getLayerNames()
    try:
        output_layers = [layer_names[i-1] for i in net.getUnconnectedOutLayers()]
    except:
        output_layers = [layer_names[i-1] for i in net.getUnconnectedOutLayers()]
    return output_layers
def draw_prediction(img, class_id, confidence, x, y, x_plus_w, y_plus_h):
    label = str(classes[class_id])
    color = COLORS[class_id]
```

```

        cv2.rectangle(img, (x,y),(x_plus_w, y_plus_h), label, color,2)
        cv2.putText(img, label, (x-10,y-10),cv2.FONT_HERSHEY_COMPLEX, color, 2)
# read input image
image = cv2.imread(args.image)
Width = image.shape[1]
Height = image.shape[0]
scale = 0.00392
classes = None
# read class names from text file
classes = None
with open(args.classes, 'r') as f:
    classes = [line.strip() for line in f.readlines()]
# generate different colors for different classes
COLORS = np.random.uniform(0, 255, size=(len(classes), 3))
# read pre-trained model and config file
net = cv2.dnn.readNet(args.weights, args.config)
# create input blob
blob = cv2.dnn.blobFromImage(image, scale, (416,416), (0,0,0), True, crop=False)
# set input blob for the network
net.setInput(blob)
# function to get the output layer names in the architecture
# function to draw bounding box on the detected object with class name
def draw_bounding_box(img, class_id, confidence, x, y, x_plus_w, y_plus_h):
    label = str(classes[class_id])
    color = COLORS[class_id]
    cv2.rectangle(img, (x,y), (x_plus_w,y_plus_h), color, 2)
    cv2.putText(img, label, (x-10,y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
# run inference through the network and gather predictions from output layers
outs = net.forward(get_output_layers(net))
# initialization
class_ids = []
confidences = []
boxes = []
conf_threshold = 0.5
nms_threshold = 0.4
# for each detection from each output layer get the confidence, class id, bounding box params
# and ignore weak detections (confidence < 0.5)
for out in outs:
    for detection in out:
        scores = detection[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]
        if confidence > 0.5:
            center_x = int(detection[0] * Width)
            center_y = int(detection[1] * Height)
            w = int(detection[2] * Width)
            h = int(detection[3] * Height)

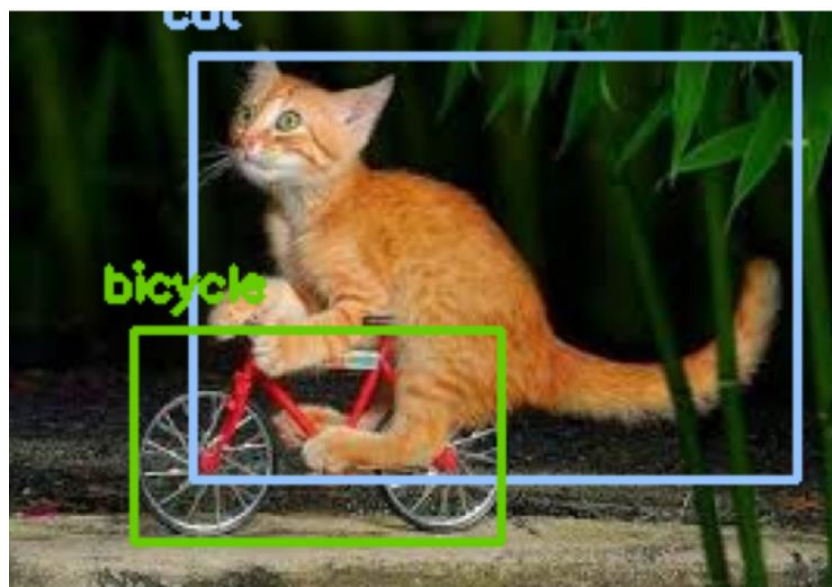
```

```

    x = center_x - w / 2
    y = center_y - h / 2
    class_ids.append(class_id)
    confidences.append(float(confidence))
    boxes.append([x, y, w, h])
# apply non-max suppression
indices = cv2.dnn.NMSBoxes(boxes, confidences, conf_threshold, nms_threshold)
# go through the detections remaining
# after nms and draw bounding box
for i in indices:
    try:
        box=boxes[i]
    except:
        i=i[0]
        box=boxes[i]
    x = box[0]
    y = box[1]
    w = box[2]
    h = box[3]
    draw_bounding_box(image, class_ids[i], confidences[i], round(x), round(y), round(x+w),
round(y+h))
# display output image
#cv2.imshow("object detection", image)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis('off') # turn off axis labels
plt.show()
# wait until any key is pressed #cv2.waitKey() # save output image to disk
#cv2.imwrite("object-detection.jpg", image) # release resources #cv2.destroyAllWindows()

```

**Output:**



## **Aim: - Perform Pedestrian Detection**

### **Theory:**

Pedestrian detection is a very important area of research because it can enhance the functionality of a pedestrian protection system in Self Driving Cars. We can extract features like head, two arms, two legs, etc, from an image of a human body and pass them to train a machine learning model. After training, the model can be used to detect and track humans in images and video streams. However, OpenCV has a built-in method to detect pedestrians. It has a pre-trained haarcascade fullbody model to detect pedestrians in images and video streams.

**Haar Cascade:** Haar cascade is an algorithm that can detect objects in images, irrespective of their scale in image and location.

This algorithm is not so complex and can run in real-time. We can train a haar-cascade detector to detect various objects like cars, bikes, buildings, fruits, etc.

Haar cascade uses the cascading window, and it tries to compute features in every window and classify whether it could be an object.

There are some pre-trained haar cascade models like:

- Human face detection
- Eye detection
- Nose / Mouth detection
- Vehicle detection
- Full body detection

In this practical we will be using the full body detection pre-trained model.

### **Code:**

```
import cv2
import matplotlib.pyplot as plt
# Initialize the Haar Cascade pedestrian detection model
pedestrian_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_fullbody.xml')
# Load the video
video_path = 'D:/MSCIT_books/sem2/vid1.mp4'
cap = cv2.VideoCapture(video_path)
# Check if the video is opened successfully
if not cap.isOpened():
    print("Error: Unable to open the video")
    exit()
# Loop through the video frames
while True:
    # Read a frame from the video
    ret, frame = cap.read()
    if not ret:
        break # Break the loop if no frame is captured
    # Convert the frame to grayScale
    gray_frame = cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)
    # Detect pedestrians in the grayscale frame
```

```
pedestrians = pedestrian_cascade.detectMultiScale(gray_frame, scaleFactor=1.1,
minNeighbors=5, minSize=(30,30))
# Draw rectangle around the detected pedestrians
for (x,y,w,h) in pedestrians:
    cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)

# Display the resulting frame with pedestrians highlighted using Matplotlib
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.axis('off') # Turn off the axis
plt.show()
# Check if the loop should be exited
# In Jupyter Notebook you might need to manually interrupt the kernel to stop the loop
# Release the video capture object
cap.release()
```

**Output:**



## **Aim: - Perform Face Recognition**

### **Theory:**

Facial recognition is a way of identifying or confirming an individual's identity using their face. Facial recognition systems can be used to identify people in photos, videos, or in real-time. Facial recognition is a category of biometric security. This is done by analyzing and comparing patterns in the image or video of the person's face to a database of known faces.

Automatic face recognition is all about extracting those meaningful features from an image, putting them into a useful representation and performing some kind of classification on them. Face recognition based on the geometric features of a face is probably the most intuitive approach to face recognition.

Facial recognition systems are a great alternative to plastic cards, password systems, attendance registers, and manual processes, and the technology represents an upgrade to secure systems access from mobile devices.

### **Code:**

```
!pip install face_recognition

import face_recognition

from matplotlib.patches import Rectangle

import matplotlib.pyplot as plt

# Load a sample picture and learn how to recognize it.

known_image = face_recognition.load_image_file("virat.jpeg") # Replace with your known
image path

known_face_encoding = face_recognition.face_encodings(known_image)[0]

# Load an unknown image to recognize faces in

unknown_image = face_recognition.load_image_file("virushka.jpeg") # Replace with your
unknown image path

unknown_face_locations = face_recognition.face_locations(unknown_image)

unknown_face_encodings = face_recognition.face_encodings(unknown_image,
unknown_face_locations)

# Convert the image to RGB (matplotlib expects RGB format)

unknown_image_rgb = unknown_image[:, :, :- 1]

# Create a subplot

fig, ax = plt.subplots(figsize=(8, 6))

ax.imshow(unknown_image_rgb)

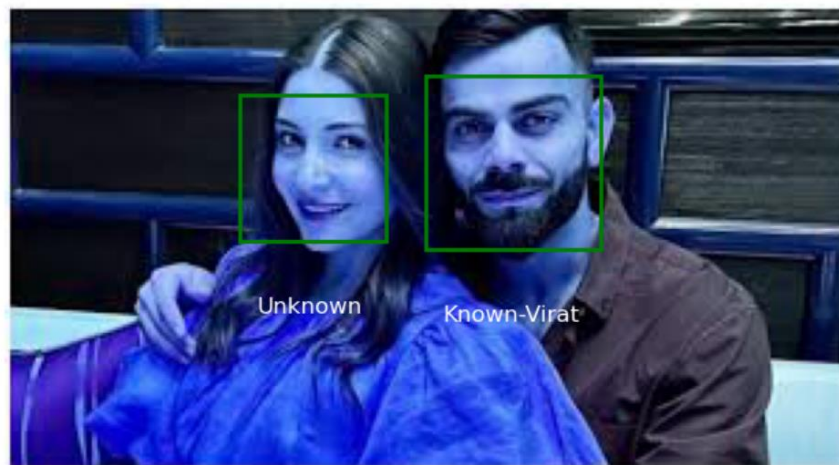
# Loop through each face found in the unknown image
```

```

for (top, right, bottom, left), unknown_face_encoding in zip(unknown_face_locations,
unknown_face_encodings):
    # See if the face ista match for the known face(s)
    results = face_recognition. compare_faces([known_face_encoding],
unknown_face_encoding)
    if results[0]:
        name = "Known-Virat"
    else:
        name = "Unknown"
    # Draw a box around the face
    ax.add_patch(Rectangle((left, top), right - left, bottom - top, fill=False, color='green',
linewidth=2))
    ax.text(left + 6, bottom + 25, name, color='white', fontsize=12)
# Display the result
plt.axis('off') # Hide the axis
plt.show()

```

**Output:**



## Practical 5

**Aim: - Construct 3D model from Images.**

### Theory:

Transforming a 2D image into a 3D space using OpenCV refers to the process of converting a two-dimensional image into a three-dimensional spatial representation using the Open Source Computer Vision Library (OpenCV). OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library written in C++ with interfaces for Python, Java, and MATLAB. It provides a wide range of features for processing and evaluating pictures and movies. This transformation involves inferring the depth information from the 2D image, typically through techniques such as stereo vision, depth estimation, or other computer vision algorithms, to create a 3D model with depth perception. This process enables various applications such as 3D reconstruction, depth sensing.

### Modules Needed:

- Matplotlib: It is a plotting library for Python programming it serves as a visualization utility library, Matplotlib is built on NumPy arrays, and designed to work with the broader SciPy stack.
- Numpy: It is a general-purpose array-processing package. It provides a high-performance multidimensional array and matrices along with a large collection of high-level mathematical functions.
- mpl\_toolkits: It provides some basic 3d plotting (scatter, surf, line, mesh) tools.

### Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def extract_keypoints_and_descriptors(image):
    sift = cv2.SIFT_create()
    keypoints, descriptors = sift.detectAndCompute(image, None)
    return keypoints, descriptors
def match_keypoints(desc1, desc2):
    bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)
    matches = bf.match(desc1, desc2)
    matches = sorted(matches, key=lambda x: x.distance)
    return matches
def draw_matches(img1, kp1, img2, kp2, matches):
    return cv2.drawMatches(img1, kp1, img2, kp2, matches[:50], None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

def reconstruct_3d_points(K, kp1, kp2, matches, E):
    points1 = np.float32([kp1[m.queryIdx].pt for m in matches])
    points2 = np.float32([kp2[m.trainIdx].pt for m in matches])
    _, R, t, mask = cv2.recoverPose(E, points1, points2, K)
```

```

# Triangulate points
P1 = np.hstack((np.eye(3, 3), np.zeros((3, 1))))
P2 = np.hstack((R, t))

points_4d_hom = cv2.triangulatePoints(P1, P2, points1.T, points2.T)
points_4d = points_4d_hom / np.tile(points_4d_hom[-1, :], (4, 1))
points_3d = points_4d[:3, :].T
return points_3d

# Load two images of a scene
img1 = cv2.imread('3d1.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('3d2.jpg', cv2.IMREAD_GRAYSCALE)

# Placeholder values for the camera matrix K (Students you should change it based on your
camera parameters.
fx = 1000 # Focal length in pixels
fy = 1000 # Focal length in pixels
cx = img1.shape[1] / 2 # Optical center X coordinate, assuming center of the image
cy = img1.shape[0] / 2 # Optical center Y coordinate, assuming center of the image
K = np.array([[fx, 0, cx],
              [0, fy, cy],
              [0, 0, 1]], dtype=float)

# Extract keypoints and descriptors
kp1, desc1 = extract_keypoints_and_descriptors(img1)
kp2, desc2 = extract_keypoints_and_descriptors(img2)

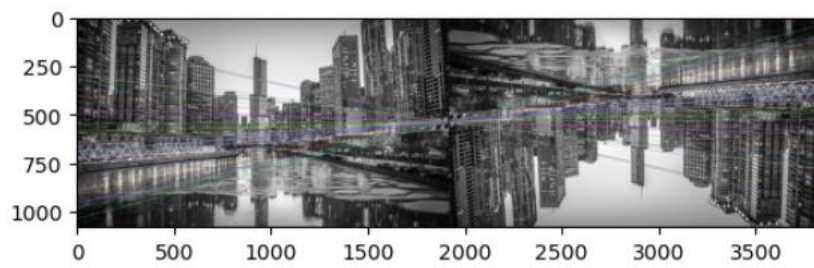
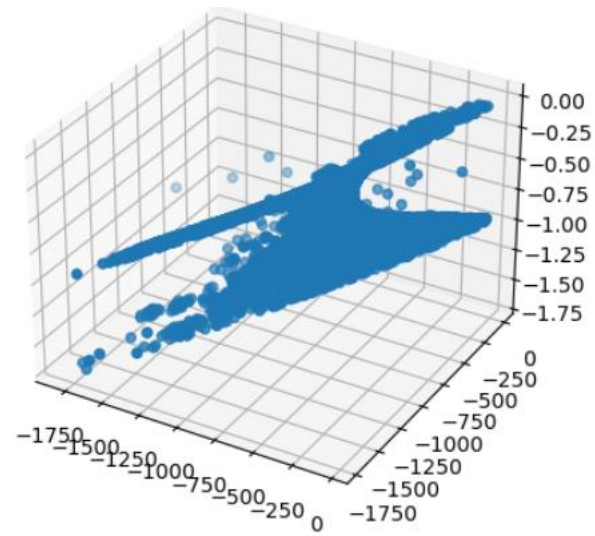
# Match keypoints
matches = match_keypoints(desc1, desc2)
# Estimate the essential matrix
points1 = np.float32([kp1[m.queryIdx].pt for m in matches])
points2 = np.float32([kp2[m.trainIdx].pt for m in matches])
F, mask = cv2.findFundamentalMat(points1, points2, cv2.FM_RANSAC)
E = K.T @ F @ K

# Reconstruct 3D points
points_3d = reconstruct_3d_points(K, kp1, kp2, matches, E)
# Visualize 3D points
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_3d[:, 0], points_3d[:, 1], points_3d[:, 2])
plt.show()

# Display matched keypoints
matched_img = draw_matches(img1, kp1, img2, kp2, matches)
plt.imshow(matched_img)
plt.show()

```

**Output:**



## Practical 6

### Aim: -Implement object detection and tracking from video

#### Theory:

Object detection, within computer vision, involves identifying objects within images or videos. Object detection is merely to recognize the object with bounding box in the image, were in image classification. Object detection applications can be performed using two different data analysis techniques:

A. Image Processing: This approach typically involves applying a series of image processing techniques, such as filtering, thresholding, and segmentation, to extract regions of interest in the image. These regions are then analysed to determine whether they contain an object or not.

B. Deep Neural Network: In a deep neural network, the algorithm is trained on large, labelled datasets to detect objects. This approach involves using a convolutional neural network (CNN) architecture, which is a type of deep learning algorithm specifically designed for image analysis.

#### Code:

```
import cv2
import numpy as np
from IPython.display import display, clear_output
from matplotlib import pyplot as plt
# Load YOLO
net = cv2.dnn.readNet("yolov3.weights", "yolov3.cfg")
classes = []
with open("coco.names", "r") as f:
    classes = [line.strip() for line in f.readlines()]
layer_names = net.getLayerNames()
output_layers = [layer_names[i-1] for i in net.getUnconnectedOutLayers()]
# loading the video
cap = cv2.VideoCapture('video1.mp4')
try:
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        height, width, channels = frame.shape
        # Detecting objects
        blob = cv2.dnn.blobFromImage(frame, 0.00392, (416,416), (0,0,0), True, crop=False)
        net.setInput(blob)
        outs = net.forward(output_layers)
        # Information to show on the screen (class_id, confidence, bounding boxes)
        class_ids = []
        confidences = []
        boxes = []
        for out in outs:
            for detection in out:
```

```

scores = detection[5:]
class_id = np.argmax(scores)
confidence = scores[class_id]
if confidence > 0.5:
    # Object Detected
    center_x = int(detection[0]*width)
    center_y = int(detection[1]*height)
    w = int(detection[2]*width)
    h = int(detection[3]*height)
    # Rectangle coordinates
    x = int(center_x - w/2)
    y = int(center_y - h/2)
    boxes.append([x, y, w, h])
    confidences.append(float(confidence))
    class_ids.append(class_id)
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
for i in range(len(boxes)):
    if i in indexes:
        x, y, w, h = boxes[i]
        label = str(classes[class_ids[i]])
        color = (0,255,0)
        cv2.rectangle(frame, (x,y), (x+w,y+h), color, 2)
        cv2.putText(frame, label, (x,y+30), cv2.FONT_HERSHEY_PLAIN, 1, color, 2)
# Convert the frame to RGB and display it
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(10,10))
plt.imshow(frame_rgb)
plt.axis('off')
display(plt.gcf())
clear_output(wait=True)
plt.close()
finally:
    cap.release()
    print('Stream ended.')

```

**Output:**



## Practical 7

**Aim: - Perform Feature extraction using RANSAC**

### Theory:

Random sample consensus, or RANSAC, is an iterative method for estimating a mathematical model from a data set that contains outliers. The RANSAC algorithm works by identifying the outliers in a data set and estimating the desired model using data that does not contain outliers.

RANSAC is accomplished with the following steps:

- Randomly selecting a subset of the data set
- Fitting a model to the selected subset
- Determining the number of outliers
- Repeating steps 1-3 for a prescribed number of iterations

In computer vision, RANSAC is used as a robust approach to estimate the fundamental matrix in stereo vision, for finding the commonality between two sets of points for feature-based object detection, and registering sequential video frames for video stabilization.

### Code:

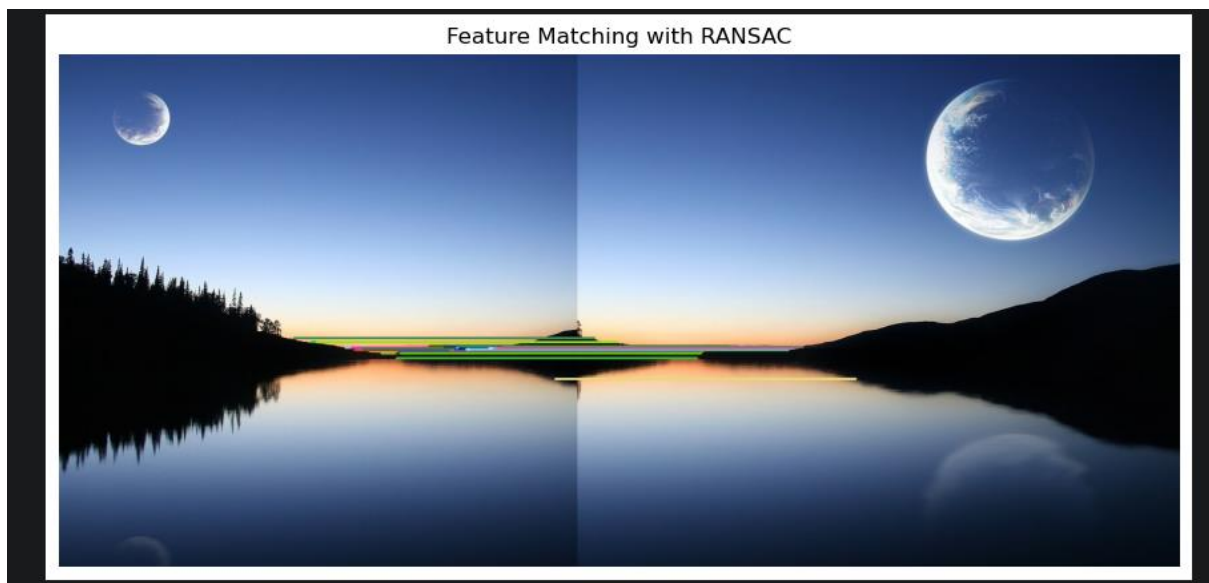
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the color images
img1_color = cv2.imread('right.jpg') # Query image (color)
img2_color = cv2.imread('left.jpg') # Train image (color)
# Convert to grayscale for feature detection
img1 = cv2.cvtColor(img1_color, cv2.COLOR_BGR2GRAY)
img2 = cv2.cvtColor(img2_color, cv2.COLOR_BGR2GRAY)
# Initialize SIFT detector
sift = cv2.SIFT_create()
# Detect keypoints and compute descriptors
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
# FLANN parameters and matcher setup
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
# Match descriptors using KNN
matches = flann.knnMatch(des1, des2, k=2)
# Ratio test to keep good matches
good = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good.append(m)
# Extract location of good matches
points1 = np.zeros((len(good), 2), dtype=np.float32)
```

```

points2 = np.zeros((len(good), 2), dtype=np.float32)
for i, match in enumerate(good):
    points1[i, :] = kp1[match.queryIdx].pt
    points2[i, :] = kp2[match.trainIdx].pt
# Find homography using RANSAC
H, status = cv2.findHomography(points1, points2, cv2.RANSAC)
# Create a new image that puts the two images side by side
height = max(img1_color.shape[0], img2_color.shape[0])
width = img1_color.shape[1] + img2_color.shape[1]
output = np.zeros((height, width, 3), dtype=np.uint8)
# Place both images within this new image
output[0:img1_color.shape[0], 0:img1_color.shape[1]] = img1_color
output[0:img2_color.shape[0],
img1_color.shape[1]:img1_color.shape[1]+img2_color.shape[1]] = img2_color
# Draw lines between the matching points
for i, (m, color) in enumerate(zip(good, np.random.randint(0, 255, (len(good), 3)))):
    if status[i]:
        pt1 = tuple(map(int, kp1[m.queryIdx].pt))
        pt2 = tuple(map(int, kp2[m.trainIdx].pt))
        pt2 = (pt2[0] + img1_color.shape[1], pt2[1]) # Shift the point for img2
        cv2.line(output, pt1, pt2, color.tolist(), 2)
# Convert the result to RGB for matplotlib display and show the final image
output_rgb = cv2.cvtColor(output, cv2.COLOR_BGR2RGB)
# Use matplotlib to display the image
plt.figure(figsize=(15, 5))
plt.imshow(output_rgb)
plt.axis('off') # Turn off axis numbers and ticks
plt.title('Feature Matching with RANSAC')
plt.show()

```

**Output:**



## Practical 8

### Aim: - Perform Colorization

#### Theory:

Image colorization is the process of taking an input grayscale (black and white) image and then producing an output colorized image that represents the semantic colours and tones of the input. In this article, we will create a program to convert a black & white image i.e grayscale image to a colour image. We are going to use the Caffe colourization model for this program. And you should be familiar with basic OpenCV functions and uses like reading an image or how to load a pre-trained model using dnn module etc. Now let us discuss the procedure that we will follow to implement the program. Like RGB, lab colour has 3 channels L, a, and b. But here instead of pixel values, these have different significances i.e:

- **L-channel:** light intensity
- **a channel:** green-red encoding
- **b channel:** blue-red encoding

And in our program, we will use the L channel of our image as input to our model to predict ab channel values and then rejoin it with the L channel to generate our final image.

#### Code:

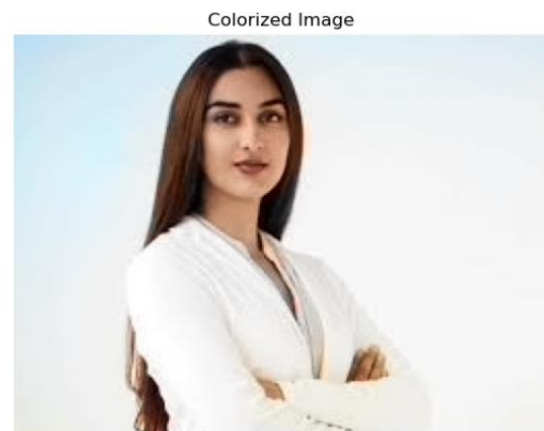
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the colorization model
net =
cv2.dnn.readNetFromCaffe('colorization_deploy_v2.prototxt','colorization_release_v2.caffe
model')
# Load cluster centers
pts_in_hull = np.load('pts_in_hull.npy', allow_pickle=True)
# Populate cluster centers as 1*1 convolution kernel
class8 = net.getLayerId('class8_ab')
conv8 = net.getLayerId('conv8_313_rh')
pts_in_hull = pts_in_hull.transpose().reshape(2,313,1,1)
net.getLayer(class8).blobs = [pts_in_hull.astype(np.float32)]
net.getLayer(conv8).blobs = [np.full([1,313],2.606, np.float32)]
# read the input image
image = cv2.imread('women1.jpeg')
# convert to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# convert to RGB
rgb_image = cv2.cvtColor(gray_image, cv2.COLOR_GRAY2RGB)
# Normalize the image
normalized_image = rgb_image.astype('float32') / 255.0
# Convert image to lab
lab_image = cv2.cvtColor(normalized_image, cv2.COLOR_RGB2Lab)
# Resize the lightness channel to network input size
```

```

resized_l_channel = cv2.resize(lab_image[:, :, 0], (224, 224))
resized_l_channel -= 50 # subtract 50 from mean-centering
# predict the a and b channels
net.setInput(cv2.dnn.blobFromImage(resized_l_channel))
pred = net.forward()[0, :, :, :].transpose((1, 2, 0))
# resize the predicted 'ab' image to the same dimensions as our input image
pred_resized = cv2.resize(pred, (image.shape[1], image.shape[0]))
# concentrate the original L channel with the predicted 'ab' channel
colorized_image = np.concatenate((lab_image[:, :, 0][:, :, np.newaxis], pred_resized), axis=2)
# convert the output image from lab to bgr
colorized_image = cv2.cvtColor(colorized_image, cv2.COLOR_Lab2BGR)
# clip any values that fall outside the range [0,1]
colorized_image = np.clip(colorized_image, 0, 1)
# convert the image to 8 bit and save
colorized_image = (255 * colorized_image).astype('uint8')
cv2.imwrite('D:/MSCIT_books/sem2/CV_pracs/colorized_image.jpg', colorized_image)
# convert the colorized image from BGR to RGB for matplotlib display
colorized_image_rgb = cv2.cvtColor(colorized_image, cv2.COLOR_BGR2RGB)
# show both the original grayscale and colorized image using matplotlib
plt.figure(figsize=(14, 7))
plt.subplot(1, 2, 1)
plt.imshow(gray_image, cmap='gray')
plt.title('Original Grayscale Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(colorized_image_rgb)
plt.title('Colorized Image')
plt.axis('off')
plt.show()

```

### Output:



## Practical 9

### Aim: - Perform Text Detection and Recognition

#### Theory:

Computer vision involves extracting information from visual data and allows us to perform complex tasks such as classification, prediction, recognition, and much more. In this practical, we will look at how to detect text using Tesseract in media, a classic optical character recognition application.

#### Optical character recognition:

Optical character recognition (OCR), is a revolutionary technology that enables machines to interpret and convert images of text into machine-readable formats. It allows us to utilize the potential of printed or handwritten text. Simply put, the goal of OCR is to convert the human perception of characters and convert them into machine-encoded text. The concept of optical character recognition is used in text detection, where we aim to identify and recognize the text found within an image or a video. In text detection, our goal is to automatically compute the bounding boxes for every region of text in an image

#### Code:

```
import pytesseract
import cv2
from pytesseract import Output
# Specify the tesseract executable path
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe'

# Load the image
image = cv2.imread('quote.png')
original_image = image.copy() # Make a copy for displaying the original later
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Use Opencv to find text blocks (simple thresholding)
_, thresh = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY_INV)

# Dilate to connect text characters
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))
dilate = cv2.dilate(thresh, kernel, iterations = 3)

# Find Contours
contours, _ = cv2.findContours(dilate, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

# Sort Contours by their y-coordinate first, then x-coordinate
lines = sorted(contours, key=lambda ctr:
(cv2.boundingRect(ctr)[1],cv2.boundingRect(ctr)[0]))

# Go through each contour, crop and read the text
for contour in lines:
```

```

x,y,w,h = cv2.boundingRect(contour)
# Make sure the contour area is a reasonable size
if w*h > 50:
    roi = image[y:y+h, x:x+w]
    text = pytesseract.image_to_string(roi, lang='eng', config= '--psm 6')
    cv2.putText(image, text, (x,y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.3, (0,255,0), 1)
    cv2.rectangle(image, (x,y), (x+w,y+h), (0,255,0), 1)
    print(text)

import matplotlib.pyplot as plt
original_image_rgb = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display the original image
plt.figure(figsize=(10,10))
plt.subplot(1,2,1)
plt.imshow(original_image_rgb)
plt.title('Original Image')
plt.axis('off')

# Display the image with detected text
plt.subplot(1,2,2)
plt.imshow(image_rgb)
plt.title('Image With Text')
plt.axis('off')

plt.show()

```

### Output:

```

BIG
JOURNEYS
BEGIN WITH

SMALL STEPS.

@MOTIVATEAMAZEBEGREAT.COM

```



## Practical 10

### Aim: - Perform Image matting and compositing

**Theory:** Image Matting is the process of accurately estimating the foreground object in images and videos. It is a very important technique in image and video editing applications, particularly in film production for creating visual effects.

In case of image segmentation, we segment the image into foreground and background by labelling the pixels. Image segmentation generates a binary image, in which a pixel either belongs to foreground or background. However, Image Matting is different from the image segmentation, wherein some pixels may belong to foreground as well as background, such pixels are called partial or mixed pixels.

In order to fully separate the foreground from the background in an image, accurate estimation of the alpha values for partial or mixed pixels is necessary.

Compositing is the process or technique of combining visual elements from separate sources into single images, often to create the illusion that all those elements are parts of the same scene.

In the matting process, a foreground element of arbitrary shape is extracted from a background image. The matte extracted by this process describes the opacity of the foreground element at every point. In the compositing process, the foreground element is placed over a new background image, using the matte to hold out those parts of the new background that the foreground element obscures.

#### Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt # Function to create a mask for the foreground
def create_mask_for_foreground(fg_image) :
    # Convert to HSV (Hue, Saturation, Value) color space for easier color segmentation
    fg_hsv = cv2.cvtColor(fg_image, cv2.COLOR_BGR2HSV)
    # Define the range of green color in HSV
    lower_green = np.array([36, 25, 25])
    upper_green = np.array([86, 255, 255])
    # Threshold the HSV image to get only green colors
    mask = cv2.inRange(fg_hsv, lower_green, upper_green)
    # Invert the mask to get the foreground
    foreground_mask = cv2.bitwise_not(mask)
    # Apply a series of dilations and erosions to remove any small blobs Left in the mask (noise)
    kernel = np.ones((3, 3), np.uint8)
    foreground_mask = cv2.morphologyEx(foreground_mask, cv2.MORPH_OPEN,
    kernel, iterations=2)
    foreground_mask = cv2.dilate(foreground_mask, kernel, iterations=4)
    return foreground_mask
# Function to apply mask to foreground and composite with new background
def composite_fg_with_new_bg(fg_image, bg_image, fg_mask):
    # Resize the background to match the size of the foreground
    bg_resized = cv2.resize(bg_image, (fg_image.shape[1], fg_image.shape[0]))
    # Normalize the mask to be in the range [0, 1] for blending
    fg_mask_normalized = fg_mask / 255.0
    # Prepare the foreground and background for blending
```

```

    fg_prepared = cv2.bitwise_and(fg_image, fg_image, mask=fg_mask)
    bg_prepared = cv2.bitwise_and(bg_resized, bg_resized,
mask=cv2.bitwise_not(fg_mask))
    # Composite the images by adding them together
    composite_image = cv2.add(fg_prepared, bg_prepared)
    return composite_image
# Load the foreground image (with a green screen background)
foreground_image_path = 'img1.jpg'
foreground = cv2.imread(foreground_image_path)
if foreground is None:
    raise ValueError("Error loading the foreground image")
# Load a new background image
background_image_path = 'background.jpg' # Replace with actual path if available
background = cv2.imread(background_image_path)
if background is None:
    # If the background image is not available, use a plain white background
    background = np.full(foreground.shape, 255, dtype=foreground.dtype)
# Create the mask for the foreground image
foreground_mask = create_mask_for_foreground(foreground)
# Composite the foreground with the new background
composite_image = composite_fg_with_new_bg(foreground, background, foreground_mask)
# Convert images from BGR to RGB for displaying
foreground_rgb = cv2.cvtColor(foreground, cv2.COLOR_BGR2RGB)
foreground_mask_rgb = cv2.cvtColor(foreground_mask, cv2.COLOR_GRAY2RGB)
composite_image_rgb = cv2.cvtColor(composite_image, cv2.COLOR_BGR2RGB)
# Display the original image, the mask, and the final composite image
plt.figure(figsize=(18, 6))
plt.subplot(1, 3, 1)
plt.imshow(foreground_rgb)
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(foreground_mask_rgb, cmap='gray')
plt.title('Foreground Mask')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(composite_image_rgb)
plt.title('Composite Image')
plt.axis('off')
plt.show()

```

### Output:

